Basic Network Setup and Security Controls

Implementation Report

Prepared By Group 30

Platform: Cisco Packet Tracer/Windows 11

Collaboraters:

Name	Student Number
Adenmosun Oreoluwa Michael	CYBE/2026/TC-6/0722
Olujobi Elijah Abayomi	CYBE/2026/TC-6/1008
Salami Julius	CYBE/2026/TC-6/0082
Isaac Precious	CYBE/2026/TC-6/0334
Uforo Afolabi	CYBE/2026/TC-6/0389
OKE JERRY OKPAIKWO	CYBE/2026/TC-6/0100
Olawumi Akinwumi-Adedeji	CBYE/2026/TC-6/0503
Dating Destiny Bakret	CYBE/2026/TC-6/1017
Obafemi Busayo	CYBE/2026/TC-6/0370
Alimi Promise Abdulahi	CYBE/2026/TC-6/0858
Muritala Abdulrazak	CYBE/2026/TC-6/0269
Charles Chukwunonye Ebuka	CYBE/2026/TC-6/0809
Chiamaka Isikaku	CYBE/2026/TC-6/0254
Olubunmi Oladele	CYBE/2026/TC-6/0896
Nwofia Charles Chiagozie	CYBE/2026/TC-6/0247

Executive Summary

This report documents the implementation of three core network security controls as part of a practical network security assessment. The exercise was carried out on a Windows 11 system using industry-standard tools like Windows Defender Firewall, Nmap and Wireshark.

The three controls implemented were:

1. Design and setup of a simple network – Cisco Packet Tracer
2. Firewall Rule Configuration – Windows Defender Firewall
3. Port Scanning and Hardening – Nmap/Zenmap
4. Network Packet Capture and Analysis – Wireshark

For the rule configurations, each control was tested via command prompt on Windows and verified with documented evidence.

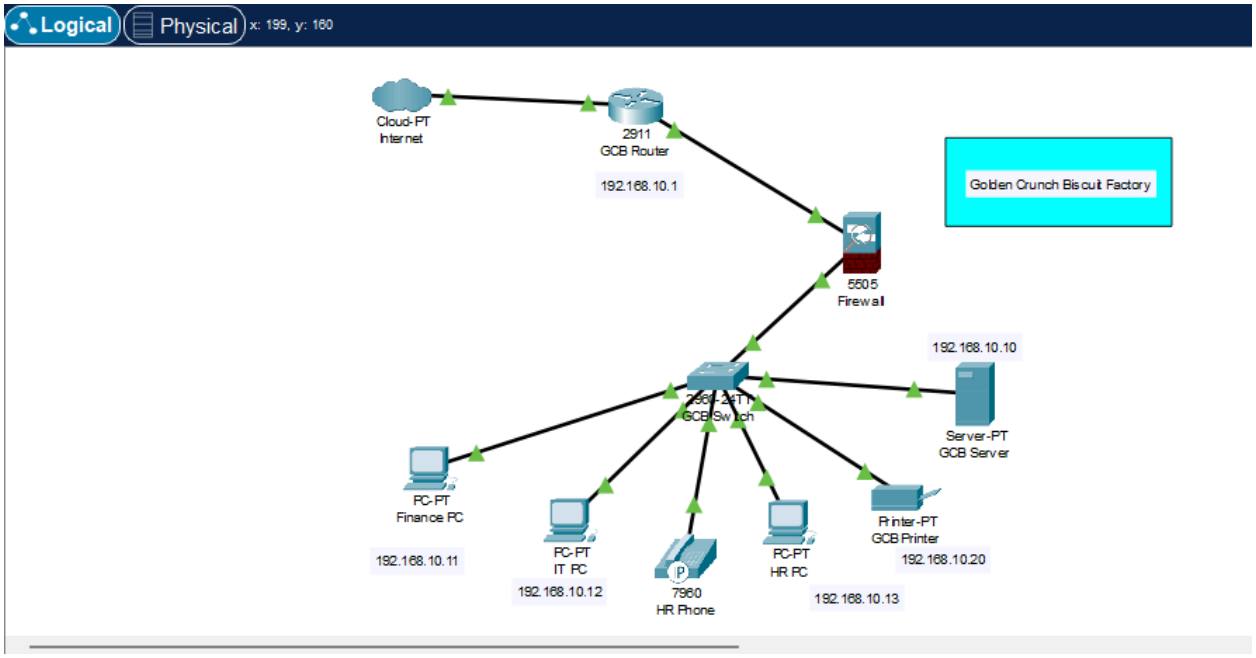
Section 1: Building a small network

The first key deliverable in this project requested a simple network setup that comprised of a router, firewall, about 3 devices, an IOT device and an Internet connection. We strategically set up an easy-to-understand network blueprint for Golden Crunch Biscuit Company using a router that's connected to the internet, a firewall positioned between the router and switch to enforce security policies between the internal and external networks. Additionally, one 2960 switch provided connectivity to end devices, which consist of three PC workstations, a printer, server and IP phone. All internal devices reside within the 192.168.1.0/24 subnet, using statically assigned IP addresses.

This design seeks to achieve the following goal;

- ➔ Provide reliable LAN connectivity for office workstations, printers, and Ip phones
- ➔ Simulate internet access via an external cloud device
- ➔ Enforce network security using a dedicated firewall with ACL rules
- ➔ Maintain a simple, scalable and easy-to-troubleshoot design.

The network topology is simple, with a layout that follows a hierarchical model, with an internet-facing router and a firewall logically separating the internal LAN zones.



Firewall Rule

An access control list named 'outside-in' was applied to the firewalls outside interface in the inbound direction. The policy was designed to deny telnet port 23 from any external source while permitting all other IP traffic.

The rationale behind creating this rule was simple: Telnet transmits credentials in plaintext and is considered a significant security risk. Blocking it at the perimeter is a recommended best practice in the industry.

ACL Commands Applied

```
enable
conf t
access-list OUTSIDE-IN deny tcp any any eq 23
access-list OUTSIDE-IN permit ip any any
```

In addition, port 23 operates over encrypted TCP sessions, which makes it vulnerable to packet sniffing and man-in-the-middle attacks. By denying inbound Telnet at the firewall, the network prevents any external party from initiating a telnet session to any internal device. This means that all legitimate remote management should rather use SSH (port 22) which encrypts each session.

```
ciscoasa>enable
Password:
ciscoasa#conf t
ciscoasa(config)#access-list OUTSIDE-IN deny tcp any any eq 23
ciscoasa(config)#access-list OUTSIDE-IN permit ip any any
ciscoasa(config)#show access-list
access-list cached ACL log flows: total 0, denied 0 (deny-flow-max 4096) alert-interval
300
access-list OUTSIDE-IN; 2 elements; name hash: 0xbb165da8
access-list OUTSIDE-IN line 1 extended deny tcp any any eq telnet(hitcnt=0) 0x0ccd3ec4
access-list OUTSIDE-IN line 2 extended permit ip any any(hitcnt=0) 0x8866a930
ciscoasa(config)#
```

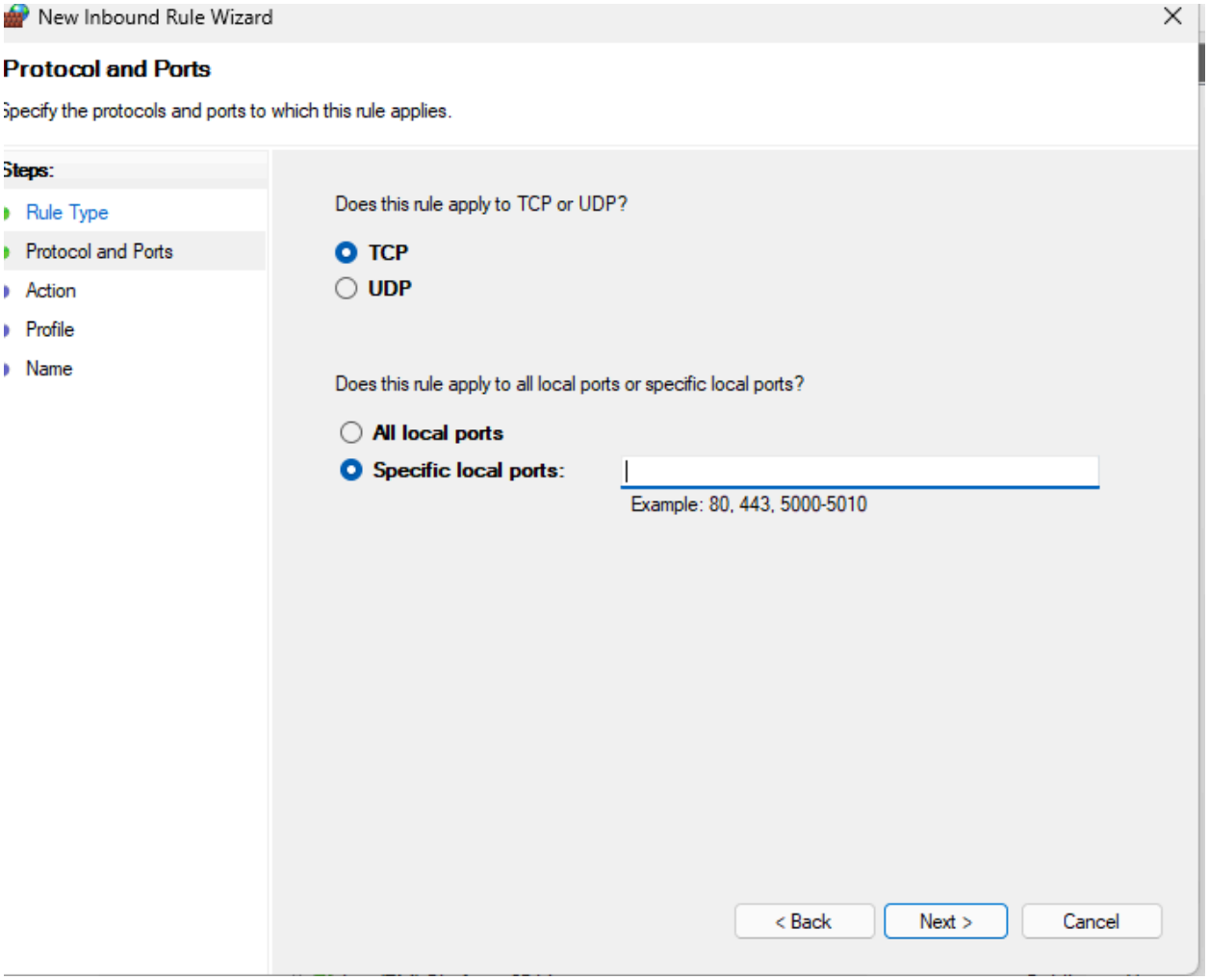
Limitations

- No VLAN segmentation: Voice and data traffic share the same subnet in this simple network design. A separate VLAN would improve security and QoS.
- Limited ACL ruleset: The current policy only blocks Telnet. A more robust design would include rules that block other unsafe protocols and implement a ‘deny-all’ at the end of each ACL.
- No redundancy: A single switch and router create single points of failure. This simply means that if there is a network breach or downtime, there is no alternative to keep systems active. Redundant links and Spanning Tree Protocol (STP) tuning would better improve this network's resilience.

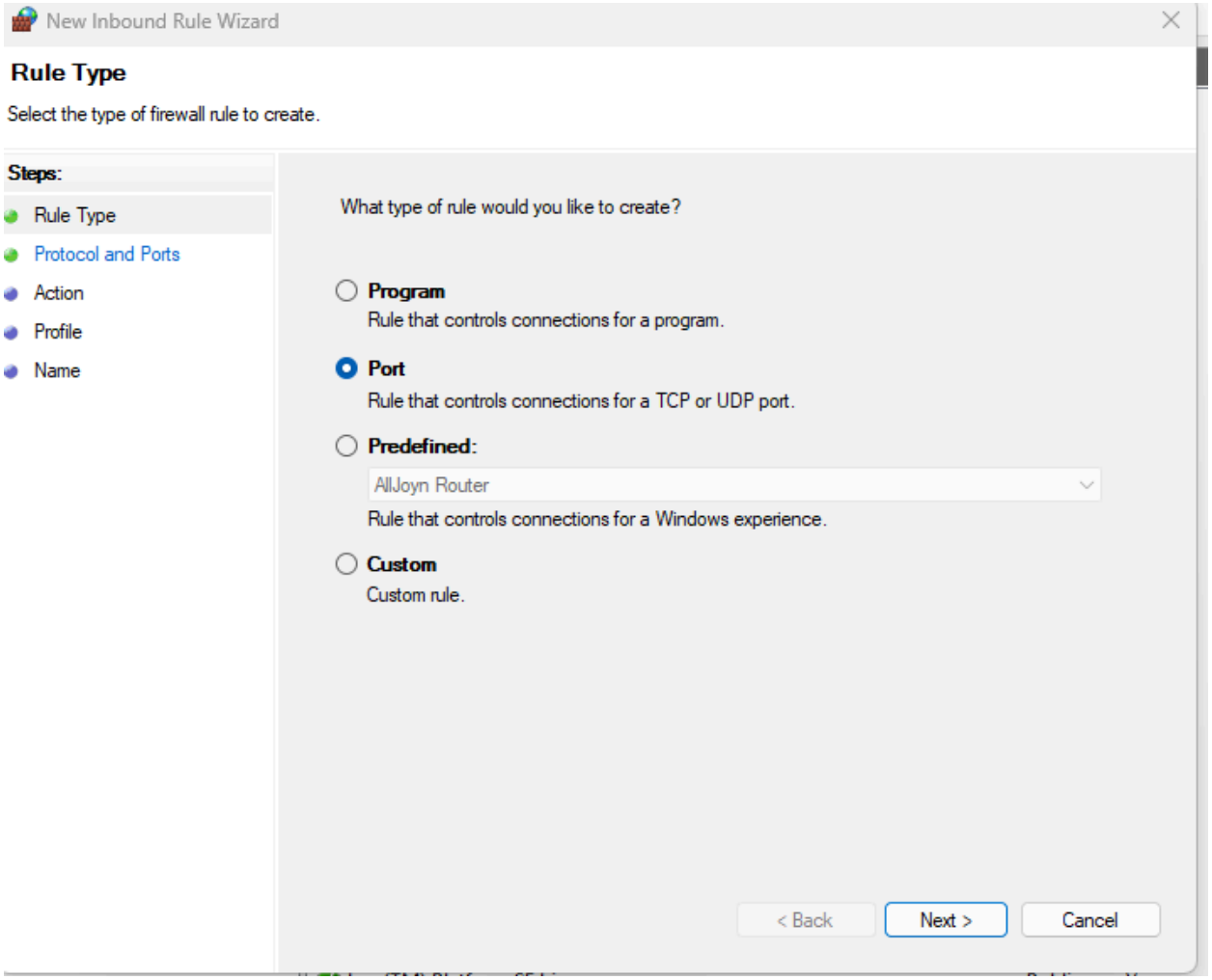
SECTION 2: Basic Network Security Controls

In this segment, we worked on using our system firewall to block or allow traffic. To achieve this, we made use of Windows Defender Firewall to create an inbound rule for TCP Port 23 (Telnet). Telnet is a legacy protocol that transmits all data in plain text with no encryption. This makes it a major security risk. Blocking port 23 prevents our local machine from accepting incoming Telnet connections.

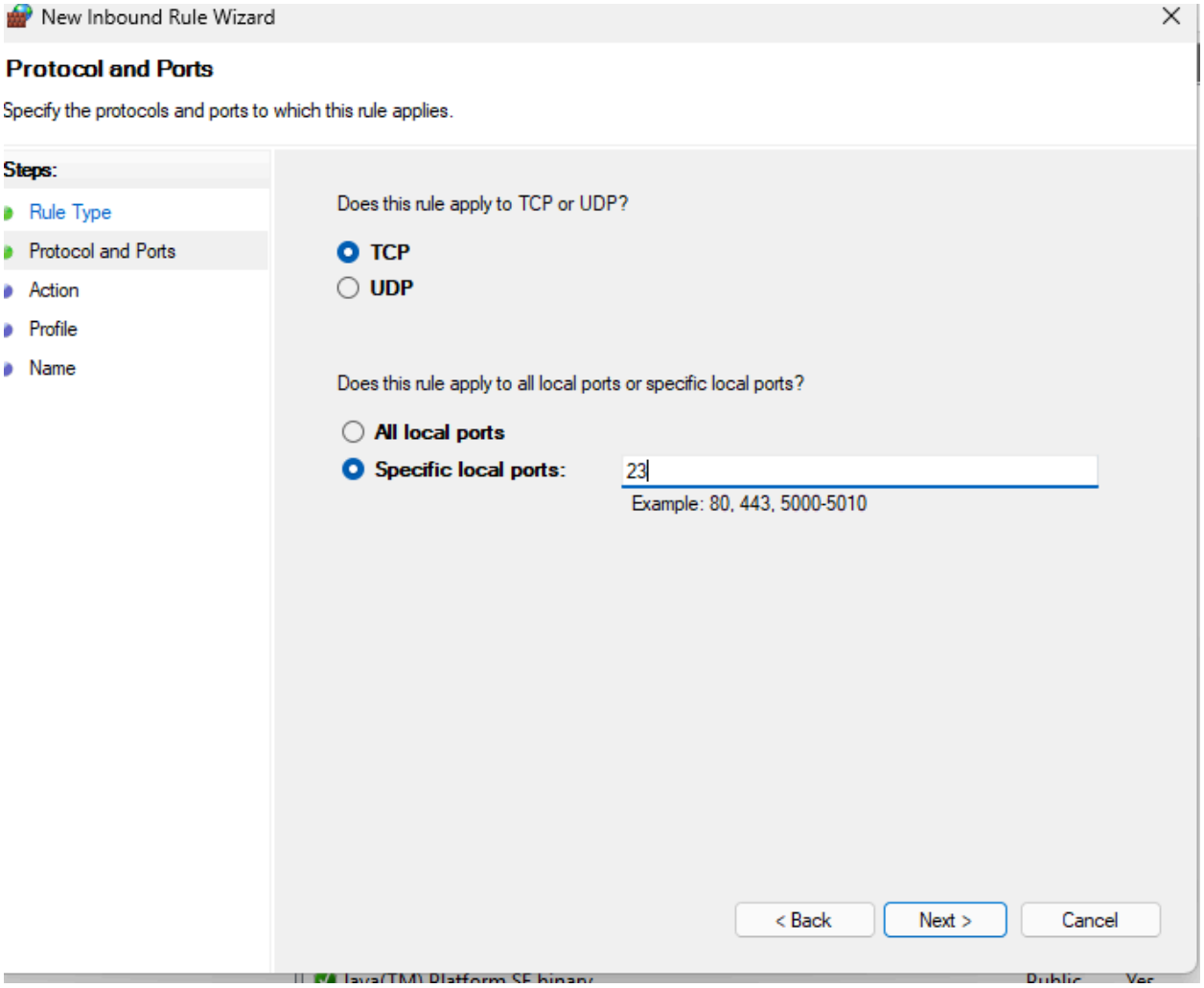
The screenshot below shows pictural evidence of creating a new rule, selecting TCP, as well as applying the rule to all specified ports.



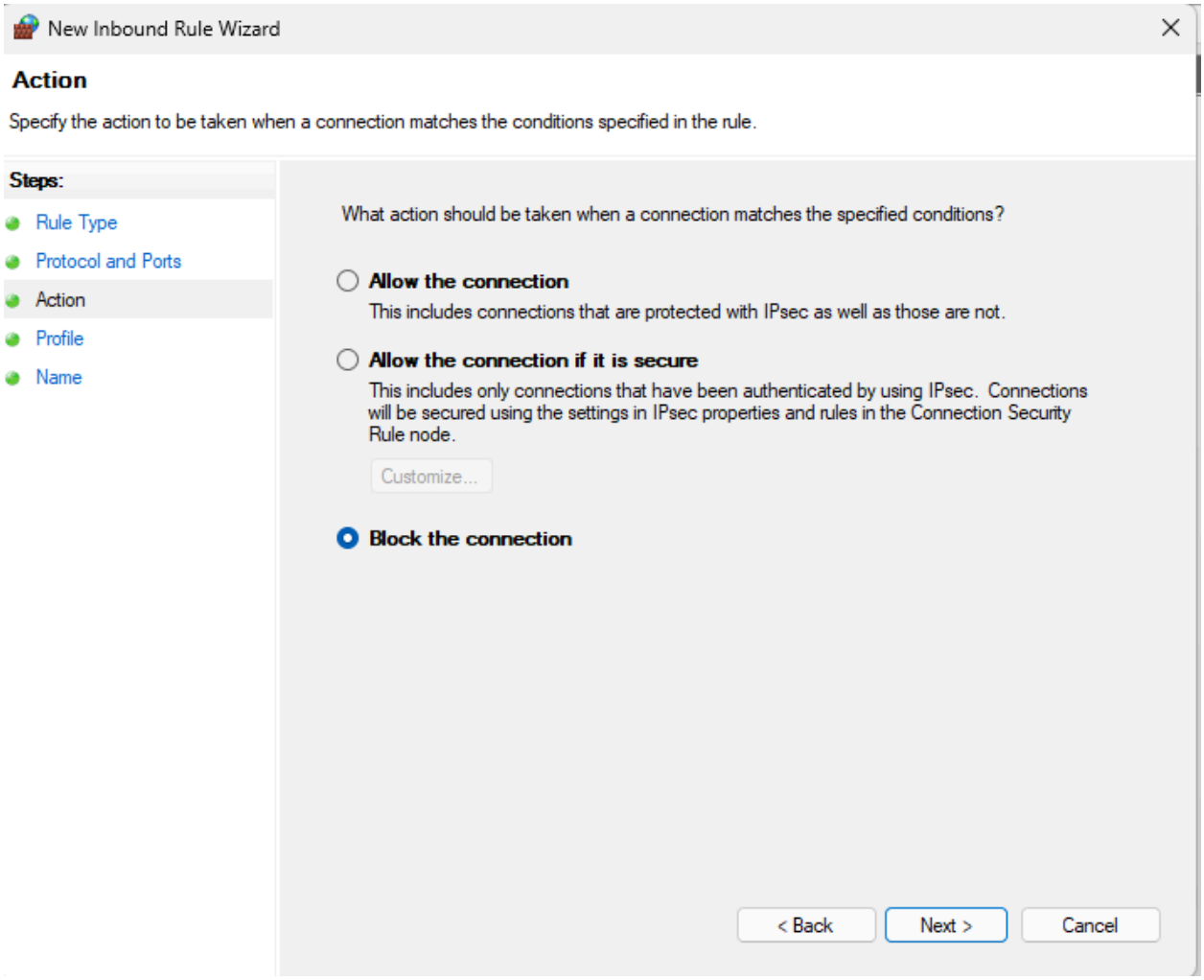
Next, we selected the type of rule we wanted to create, which was ports, as it controls the connections for TCP and UDP ports.



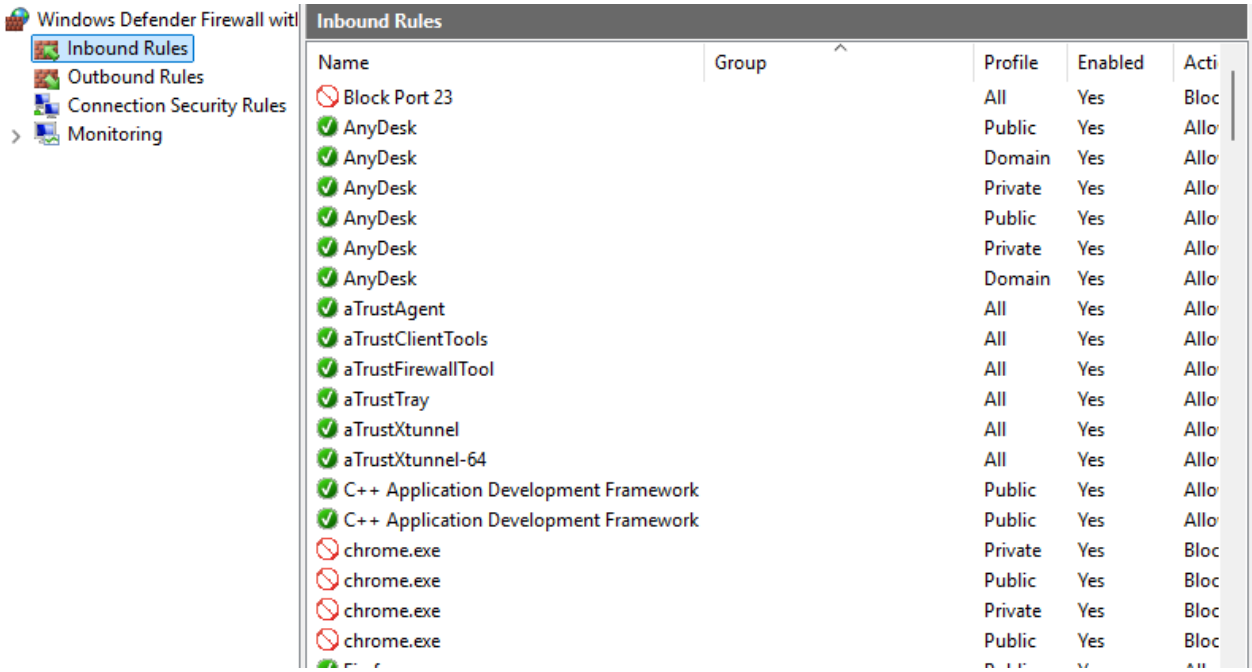
Following that, we specified the port type to block, which was TCP, and went on to apply the rule specifically to port 23.



On completing this segment, we were prompted to select if we would like to block or allow port 23. This allowed us to block the Telnet port traffic from coming in. Domain, private and public profiles were all selected to harden the rule we were creating.



The screenshot below explicitly proves that the Block Port 23 rule was created and is active. Next, we had to test if our rule was working from the local machine's terminal.



The rule was tested using PowerShell with the command Test-NetConnection shown below, targeting the localhost on port 23. Initially, there was a typo in our command, which was immediately corrected. This helped us remember the importance of spell-checking commands thoroughly before pressing Enter. The result below confirmed the rule was working correctly.

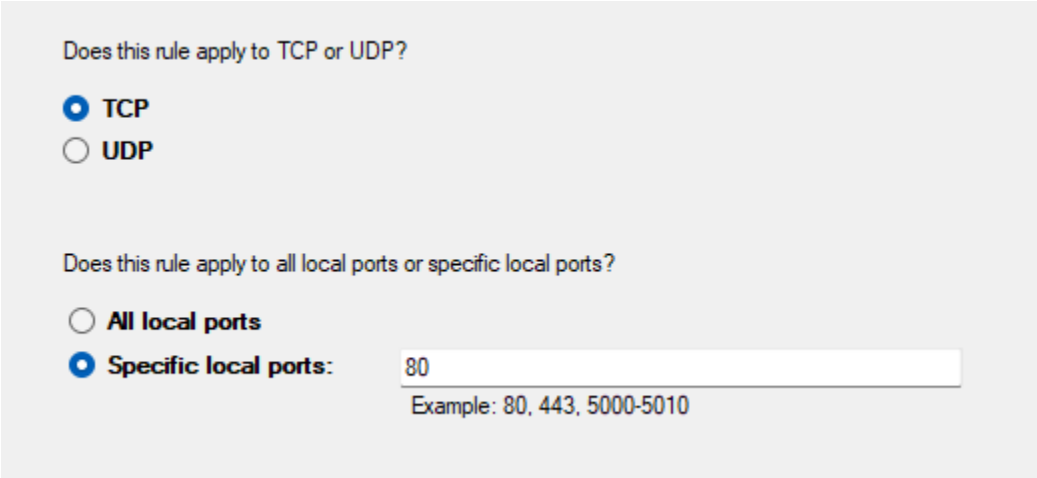
```
Test-NetConnection -ComputerName localhost -port 23.
```

Test Result Field	Value	Interpretation
ComputerName	localhost	Tested on local machine
RemotePort	23	Telnet port tested
TcpTestSucceeded	False	Port successfully blocked

```
PS C:\Users\uforo> Test-NetConnection -ComputerName localhost -Port 23
WARNING: TCP connect to (::1 : 23) failed
WARNING: TCP connect to (127.0.0.1 : 23) failed


ComputerName            : localhost
RemoteAddress           : ::1
RemotePort              : 23
InterfaceAlias          : Loopback Pseudo-Interface 1
SourceAddress           : ::1
PingSucceeded           : True
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded        : False
```

Once the first port rule was successful, we moved to create a new port, this time to allow TCP Port 80.



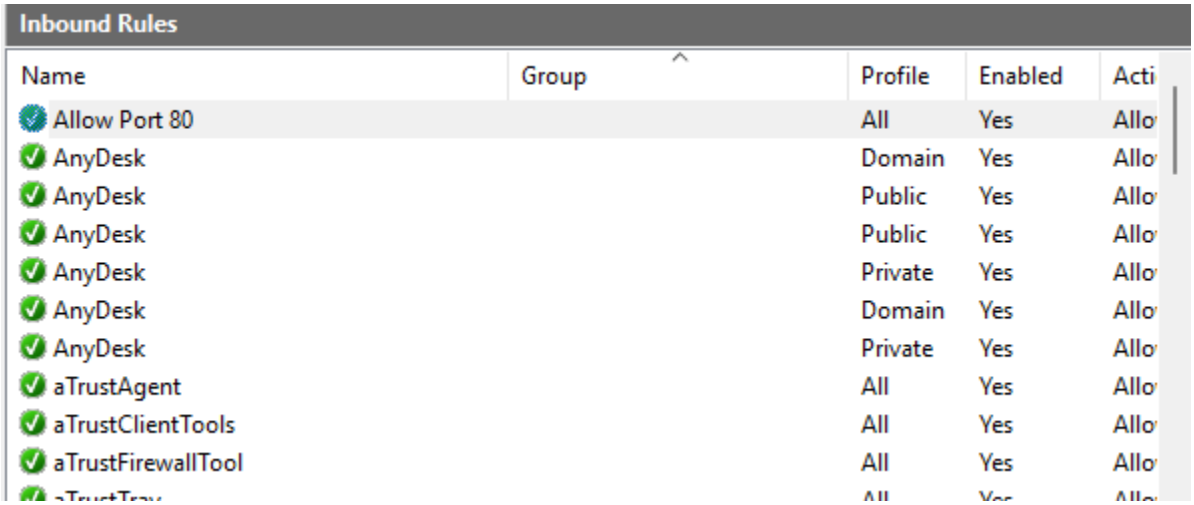
This clearly demonstrates the ability to permit traffic in addition to blocking it, showing how bidirectional firewall management and rules can be. The table below further explains the settings we put in place.

Settings	**Values**
Rule Type	Port
Protocol	TCP
Port Number	80
Action	Allow the Connection
Profiles Applied	Domain, Private, Public

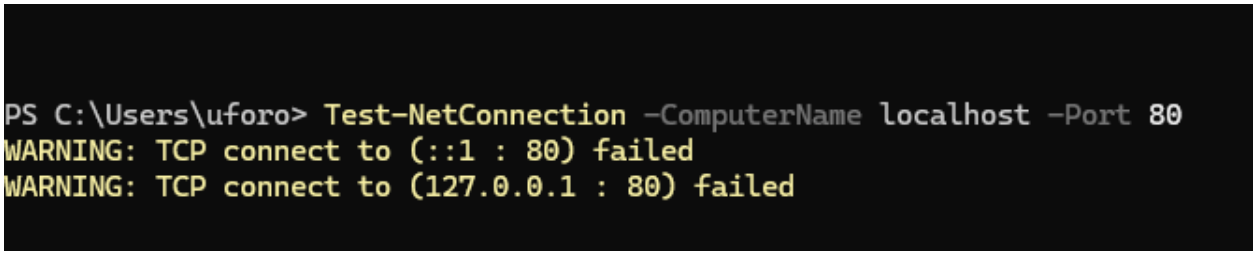
Rule Name	Allow Port 80

Testing the Allow Rule

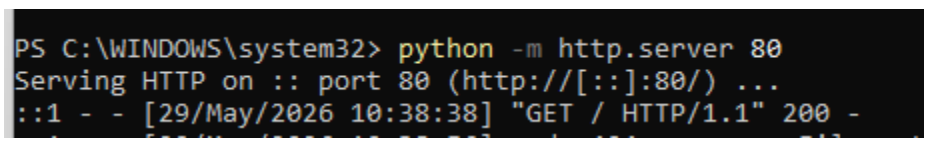
To test the allow rule, we went back to our Terminal to verify that traffic was allowed by using the site http://localhost:8080. The browser successfully loaded the directory listing page, which confirmed that HTTP traffic was permitted through the firewall.



Initially, when HTTP traffic was tested on the local host, the connection failed to load on both IPv4 localhost addresses. This showed that even though the firewall was configured to allow traffic on port 80, no web service was actually listening on it. This particular exercise showed that while firewall rules control network access, they sometimes still require active services to enable proper network communication.



The allow rule was further verified by running a Python HTTP server on port 80 and accessing it via browser at http://localhost.



The screenshot below shows the successful directory listing that was loaded, which confirmed that HTTP traffic was permitted through the firewall.

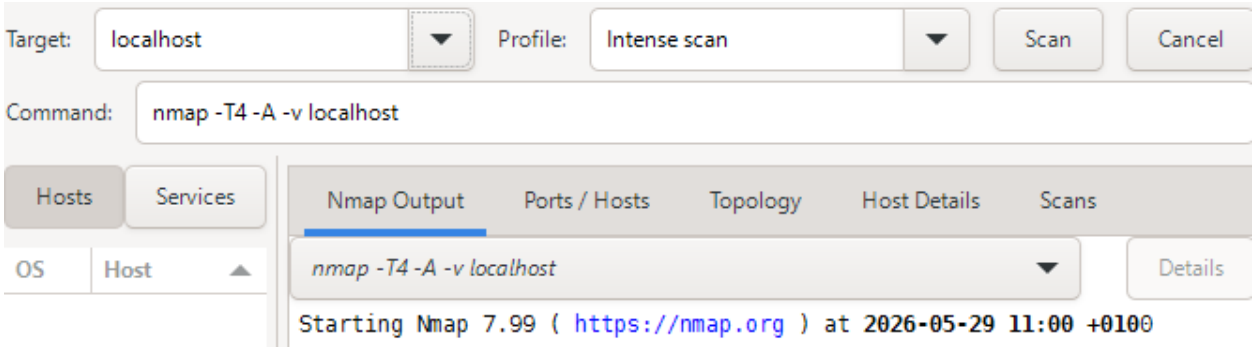
Directory listing for /

- %userprofile%/
- 0409/
- 0ae3b998-9a38-4b72-a4c4-06849441518d_Servicing-Stack.dll
- 3bc29097-7317-41d3-93b9-38a48f99d48a_mssrch.dll
- 4545ffe2-0dc4-4df4-9d02-299ef204635e_hvsocket.dll
- 5E37410B-D6F1-471D-AE27-563CEAC0D6B2
- 69fe178f-26e7-43a9-aa7d-2b616b672dde_eventlogservice.dll
- 6bea57fb-8dfb-4177-9ae8-42e8b3529933_RuntimeDeviceInstall.dll
- @AdvancedKeySettingsNotification.png
- @AppHelpToast.png
- @AudioToastIcon.png
- @AutoSrToastIcon.png
- @BackgroundAccessToastIcon.png
- @bitlockertoastimage.png
- @edptoastimage.png
- @EnrollmentToastIcon.png

2.2 Port Scanning and Hardening

In this segment, we were required to use Nmap to conduct a thorough scan of the local machine. The purpose was to identify open ports, running services and then take action to close unnecessary or risky ports.

Once Nmap was initiated, an intense scan was performed targeting localhost. The scan was completed in about 120 seconds, starting at 11:11 and finishing at 11:15.



A simple HTTP web server was deployed on the local machine using Python's built-in HTTP server module on Port 80. After starting the service, a web browser was used to access

(http://localhost), successfully displaying the directory contents hosted by the server. This confirmed that the web service was running and accepting connections on Port 80.

To verify the exposed service, Nmap was used to perform an intensive scan of the localhost system. The scan was conducted to identify open ports and detect active network services. The result confirmed that Port 80 was accessible, demonstrating how network scanning tools can discover running services and potentially exposed network resources.

```
nmap -T4 -A -v localhost
```

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-29 11:00 +0100
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 11:11
Completed NSE at 11:11, 0.05s elapsed
Initiating NSE at 11:11
Completed NSE at 11:11, 0.00s elapsed
Initiating NSE at 11:11
Completed NSE at 11:11, 0.00s elapsed
Initiating SYN Stealth Scan at 11:11
Scanning localhost (127.0.0.1) [1000 ports]
Discovered open port 80/tcp on 127.0.0.1
Discovered open port 135/tcp on 127.0.0.1
Discovered open port 445/tcp on 127.0.0.1
Discovered open port 10000/tcp on 127.0.0.1
Discovered open port 5432/tcp on 127.0.0.1
Discovered open port 912/tcp on 127.0.0.1
Discovered open port 7070/tcp on 127.0.0.1
Discovered open port 902/tcp on 127.0.0.1
Completed SYN Stealth Scan at 11:11, 6.53s elapsed (1000 total ports)
Initiating Service scan at 11:11
Scanning 8 services on localhost (127.0.0.1)
Completed Service scan at 11:14, 164.47s elapsed (8 services on 1 host)
Initiating OS detection (try #1) against localhost (127.0.0.1)
NSE: Script scanning 127.0.0.1.
Initiating NSE at 11:14
Completed NSE at 11:15, 70.43s elapsed
Initiating NSE at 11:15
Completed NSE at 11:15, 12.97s elapsed
Initiating NSE at 11:15
Completed NSE at 11:15, 0.00s elapsed
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00086s latency).
```

Nmap Port Scanning and Service Discovery

An Nmap intensive scan (nmap -T4 -A -v localhost) was performed against the local machine to identify active services and open ports. The scan successfully detected several open TCP ports, including Port 80, which had been enabled earlier via the Python HTTP server. Additional open ports such as 135, 445, 5432, and others were also identified.

The scan demonstrated how network reconnaissance tools can be used to discover services running on a host. From a security perspective, this highlights the importance of the attack surface. The results confirmed that the HTTP service was accessible and showed how attackers could potentially gather information about a system through port scanning.

Nmap Output | Ports / Hosts | Topology | Host Details | Scans

Port	Protocol	State	Service	Version
80	tcp	open	http	SimpleHTTPServer 0.6 (Python 3.13.12)
135	tcp	open	msrpc	Microsoft Windows RPC
445	tcp	open	microsoft-ds	
902	tcp	open	vmware-auth	VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912	tcp	open	ftp	vsftpd (before 2.0.8) or WU-FTPD
5432	tcp	open	postgresql	PostgreSQL DB
7070	tcp	open	realserver	
10000	tcp	open	snet-sensor-mgmt	

Nmap Service Enumeration and Security Assessment

After deploying network services, an Nmap intensive scan (nmap -T4 -A -v localhost) was performed to identify open ports and gather information about the services running on the system. The scan detected several open ports, including Port 80 (HTTP), Port 912 (FTP), Port 5432 (PostgreSQL), and other system and application-related services.

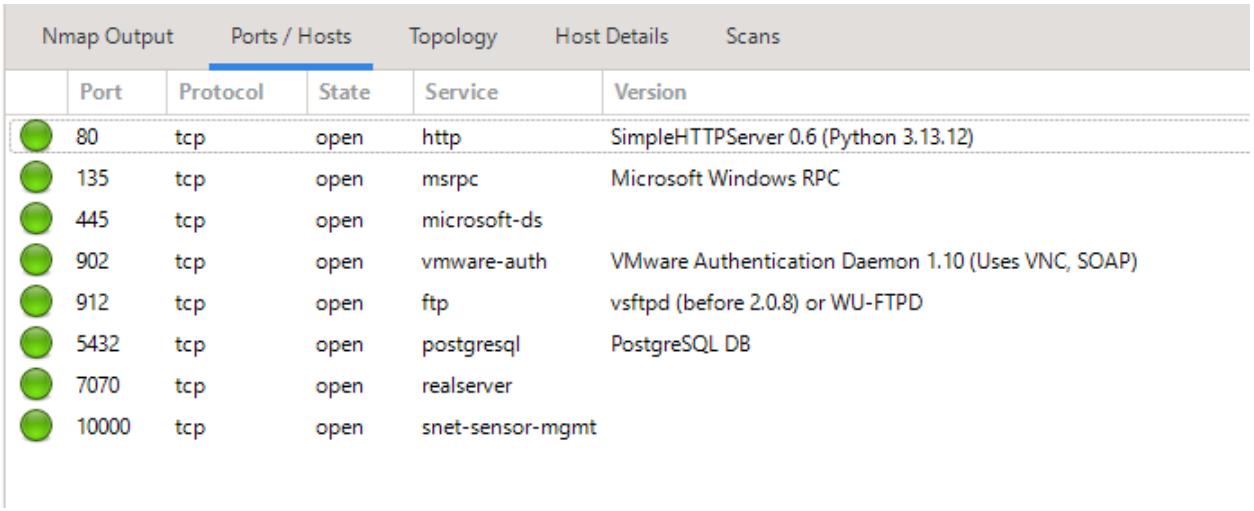
Nmap Successfully identified the service versions associated with the open ports. Port 80 was found to be running a Python SimpleHTTPServer, confirming that the web server configured earlier was active and accessible. The scan also revealed additional services such as FTP, PostgreSQL, VMware Authentication Service, and Microsoft Windows RPC.

Following the Nmap scan, Port 5432 (PostgreSQL) was identified as an open service on the system. To improve security, a Windows firewall rule was created to block inbound traffic to Port 5432.

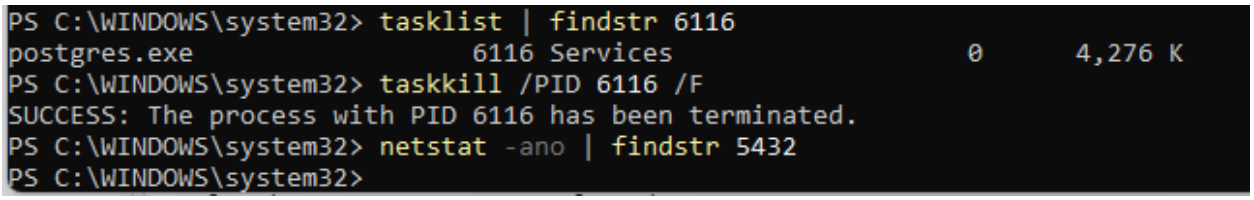
Inbound Rules

Name	Group	Profile	Enabled	Acti
Block 5432		All	Yes	Bloc
Allow Port 80		All	No	Allo
AnyDesk		Domain	Yes	Allo
AnyDesk		Public	Yes	Allo
AnyDesk		Public	Yes	Allo
AnyDesk		Private	Yes	Allo

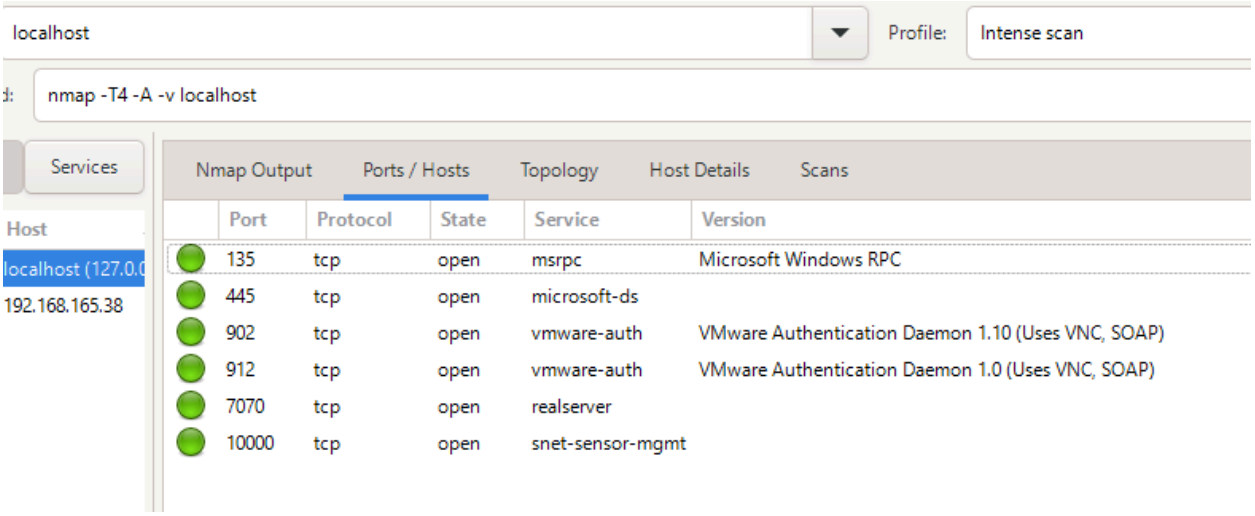
However, even after creating this rule, Port 5432 still remained open, after another scan was performed on Nmap, as seen below.



To troubleshoot this, Windows Terminal was used to identify the root service for port 5432 and terminate it.



The PostgreSQL process (postgres.exe) was then located and terminated using PowerShell commands. A final verification confirmed that the service was no longer listening on Port 5432. This activity demonstrated port hardening and attack surface reduction by disabling and restricting access to an unnecessary network service.



After applying port hardening measures, another Nmap scan was performed to verify the system's security posture. The scan confirmed that previously exposed services, including Port 80 and Port 5432, were no longer accessible, demonstrating the effectiveness of the firewall

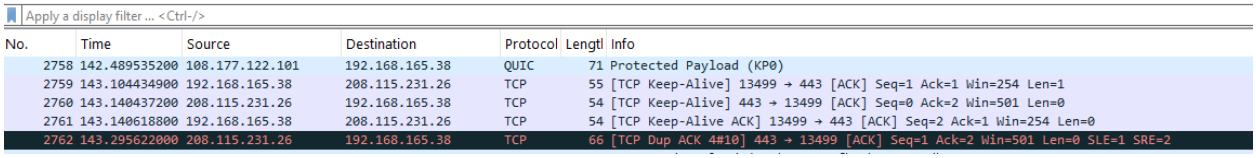
rules and services management actions. Only the remaining required services were detected during the scan.

Wireshark Capture

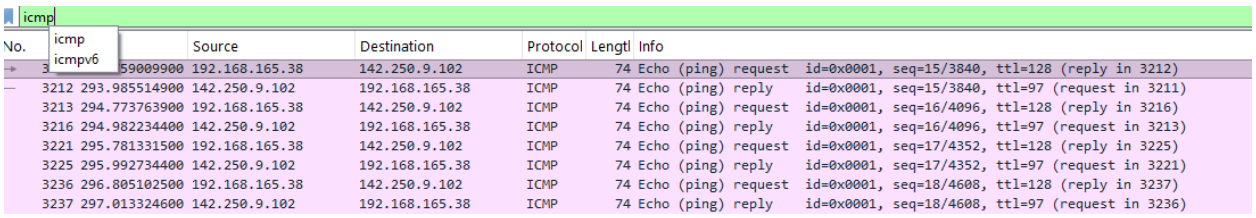
In this segment, we were required to record and capture live network traffic which was carried out on the WiFi interface. A total of four types of traffic were captured and filtered to demonstrate different network protocols and their security implications.

Capture Type	Filter Used	Purpose
HTTP	http	Show unencrypted web traffic
DNS	dns	Show domain name lookups
ICMP	icmp	Show ping/echo
HTTPS/TLS	tls	Show encrypted traffic

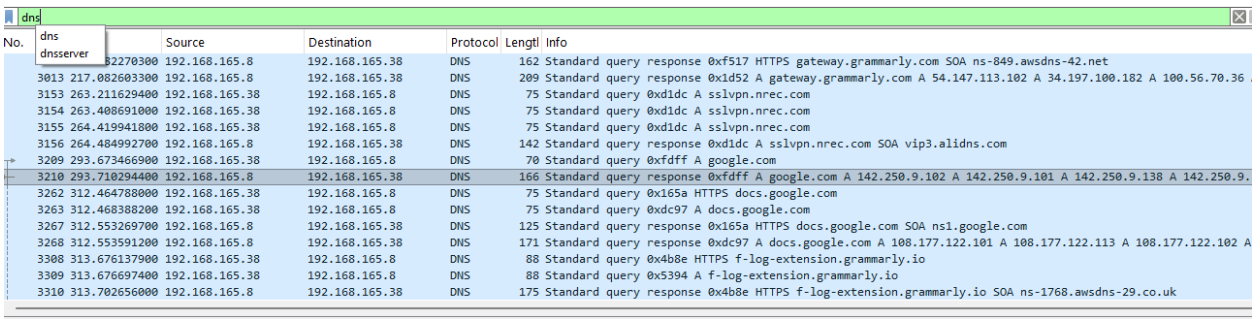
Screenshot of open Wireshark Interface



Screenshot of packet capture using icmp filter



Screenshot of packet capture using dns filter

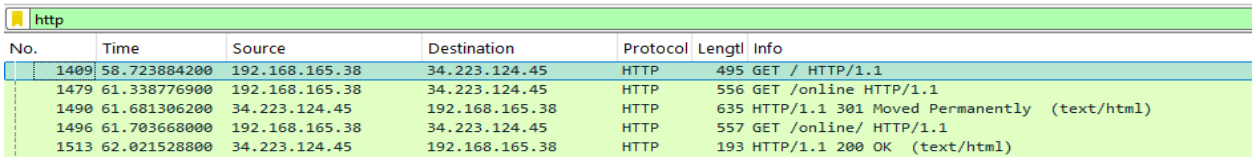


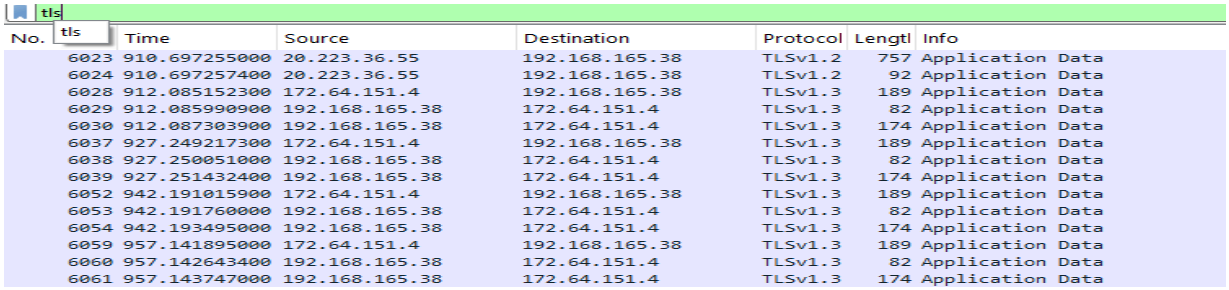
The analysis provided visibility into how devices communicate across the network. Showing connection requests, responses, and name resolution activities.

It's essential to note that HTTP traffic took a while to come up; to troubleshoot this, the site NeverSSL was used to force the Wi-Fi captive portal open.

Once the HTTP traffic was generated during web browsing, the packet capture showed HTTP GET requests from the local machine (192.168.165.38) to a web server (34.223.124.45), followed by server responses including "301 Moved Permanently" and "200 OK" status codes. The capture demonstrated how HTTP traffic can be observed and analysed in Wireshark, allowing users to view communication between a client and a web server. This highlights the importance of monitoring network traffic to understand how data is transmitted across a network.

Screenshot of packet capture using http filter.





Screenshot of TLS (HTTPS) traffic.

Wireshark TLS Traffic Analysis

Wireshark was also used to capture encrypted network traffic, TLS 1.2 and TLS 1.3. Unlike HTTP capture, the packets appeared as Application Data, meaning the actual contents of the communication could not be viewed. The capture showed secure communication between the local machine and remote servers while protecting the transmitted data from being exposed. This demonstrated how encryption helps secure network traffic and prevents sensitive information from being easily inspected by unauthorised users.

The Wireshark analysis showed the difference between standard HTTP communication and encrypted TLS communication. HTTP traffic revealed request and response information, while TLS traffic protected the data through encryption, making the packet contents unreadable.

Simulated threat and response

To simulate a common network reconnaissance attack, an Nmap scan was performed against the localhost system. The scan identified several open ports and the services running on them, including ports 135, 445, 902, 912, 7070, and 1000.

```
PS C:\WINDOWS\system32> nmap localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-29 13:37 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0015s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
445/tcp   open  microsoft-ds
902/tcp   open  iss-realsecure
912/tcp   open  apex-mesh
7070/tcp  open  realserver
10000/tcp open  snet-sensor-mgmt

Nmap done: 1 IP address (1 host up) scanned in 0.98 seconds
PS C:\WINDOWS\system32>
```

This demonstrated how an attacker can use port-scanning tools to discover accessible services and gather information about a target system. The results were then used to identify unnecessary services and apply security controls such as firewall rules and port hardening to reduce the network's attack surface.

tcp

No.	Time	Source	Destination	Protocol	Lengtl	Info
304	79.896216100	208.115.231.26	192.168.165.38	TCP	66	[TCP Keep-Alive ACK] 443 → 13499 [ACK] Seq=1 Ack=2 Win=501 Len=0 SLE=1 SRE=2
305	80.863593100	192.168.165.38	142.250.74.174	UDP	71	52432 → 443 Len=29
306	81.026184400	142.250.74.174	192.168.165.38	UDP	66	443 → 52432 Len=24
307	87.435858800	192.168.165.38	142.250.74.174	UDP	71	52432 → 443 Len=29
308	87.614474900	142.250.74.174	192.168.165.38	UDP	66	443 → 52432 Len=24
309	89.911937100	192.168.165.38	208.115.231.26	TCP	55	[TCP Keep-Alive] 13499 → 443 [ACK] Seq=1 Ack=1 Win=254 Len=1
310	89.965645200	208.115.231.26	192.168.165.38	TCP	54	[TCP Keep-Alive] 443 → 13499 [ACK] Seq=0 Ack=2 Win=501 Len=0
311	89.965844700	192.168.165.38	208.115.231.26	TCP	54	[TCP Keep-Alive ACK] 13499 → 443 [ACK] Seq=2 Ack=1 Win=254 Len=0
312	90.160623300	208.115.231.26	192.168.165.38	TCP	66	[TCP Dup ACK 34#4] 443 → 13499 [ACK] Seq=1 Ack=2 Win=501 Len=0 SLE=1 SRE=2
313	91.338997500	104.18.36.252	192.168.165.38	TLSv1.2	189	Application Data
314	91.339766000	192.168.165.38	104.18.36.252	TLSv1.2	82	Application Data
315	91.341304300	192.168.165.38	104.18.36.252	TLSv1.2	174	Application Data
316	91.555923700	104.18.36.252	192.168.165.38	TCP	54	443 → 50101 [ACK] Seq=946 Ack=917 Win=16 Len=0
317	91.555925100	104.18.36.252	192.168.165.38	TCP	54	443 → 50101 [ACK] Seq=946 Ack=1037 Win=16 Len=0

Wireshark TCP Traffic Analysis

Wireshark was used to capture and analyse TCP network traffic on the system. The packet capture showed communication between the local machine (192.168.165.38) and external hosts over TCP connections, including acknowledgements (ACKs), keep-alive messages, and encrypted TLS traffic. These packets demonstrated how devices maintain active network sessions and exchange data across the network. This exercise helped visualise real-time

network communication and showed how Wireshark can be used to monitor, analyse, and troubleshoot network traffic for security and performance purposes.

```
PS C:\WINDOWS\system32> Get-NetFirewallProfile


Name                            : Domain
Enabled                         : True
DefaultInboundAction            : NotConfigured
DefaultOutboundAction           : NotConfigured
AllowInboundRules               : NotConfigured
AllowLocalFirewallRules         : NotConfigured
AllowLocalIPsecRules            : NotConfigured
AllowUserApps                   : NotConfigured
AllowUserPorts                  : NotConfigured
AllowUnicastResponseToMulticast : NotConfigured
NotifyOnListen                  : True
EnableStealthModeForIPsec       : NotConfigured
LogFileName                     : %systemroot%\system32\LogFiles\Firewall\pfirewall.log
LogMaxSizeKilobytes             : 4096
LogAllowed                      : False
LogBlocked                      : False
LogIgnored                      : NotConfigured
DisabledInterfaceAliases        : {NotConfigured}

Name                            : Private
Enabled                         : True
DefaultInboundAction            : NotConfigured
DefaultOutboundAction           : NotConfigured
```

Firewall Configuration Verification

In response to this threat, PowerShell was used to verify the current Windows Firewall Configuration by running the (Get-NetFirewallProfile) command. The output showed that the firewall was enabled for both the Domain and Private network profiles, confirming that firewall protection was active on the system. Reviewing the firewall settings helped ensure that security controls were properly configured and that the network was protected against unauthorised inbound connections.

The result was confirmed to be active, providing an additional layer of security for the network and supporting the firewall rules implemented during the project.

```
Name                            : Public
Enabled                         : True
DefaultInboundAction            : NotConfigured
DefaultOutboundAction           : NotConfigured
AllowInboundRules               : NotConfigured
AllowLocalFirewallRules         : NotConfigured
AllowLocalIPsecRules            : NotConfigured
AllowUserApps                   : NotConfigured
AllowUserPorts                  : NotConfigured
AllowUnicastResponseToMulticast : NotConfigured
NotifyOnListen                  : True
```

This screenshot shows the status of the Windows Firewall Public Profile. The firewall is enabled, providing protection against unauthorised network access. Verifying the firewall configuration

was part of the network security controls implemented in this project. The enabled firewall helps restrict unwanted inbound connections and serves as a key defence mechanism against threats such as unauthorised access attempts and network reconnaissance activities. This control contributes to the overall security of the small network design.

Project Conclusion and Key Takeaways

This project gave the participants the opportunity to apply several core network security concepts in a practical environment using accessible industry-standard tools. A small network design was successfully set up using Cisco Packet Tracer, with firewall rules implemented to control traffic.

Port scanning and hardening were equally carried out using Nmap, while live network traffic was analysed via Wireshark.

Beyond simply completing the required tasks, the exercise helped to better exemplify how different security controls contribute to protecting a network. It was easy to see firsthand how firewall rules can restrict unwanted access, how port scanning can reveal potential security risks, and how packet analysis can provide visibility into ongoing or dormant network communications.

The project also reinforced the importance of continuous monitoring and proper network configuration while reducing unnecessary exposure to threats to improve overall security. Overall, this was a valuable, hands-on experience that strengthened the understanding of network security fundamentals and how they are applied in real-world environments.

- ❖ MTTD (Mean Time to Detect) – Target: under 12 hours.
- ❖ MTTR (Mean Time to Respond) – Target: under 8 hours.
- ❖ Financial Impact – Track ransom demand (₦15,000,000) vs. actual loss.
- Systems Restored – Number of workstations and ERP backups successfully recovered.
- Accounts Compromised – Record exact number; escalate if more than 2 accounts affected.
- Communication Timeliness – Internal updates within 2 hours, external notifications within 48–72 hours.
- Lessons Learned – Document detection gaps, staff awareness issues, and technical improvements.